OJT Weekly Report and DTR Monitoring System

Module Plan, Network Infrastructure Plan, and Test Plan

Item	Value
Revision	Rev 1
Date	2026-10-21
Course	SOFTDEV
Contents	1 Modules and Features (DL-01 Module plan) 2 Network Infrastructure (DL-02 Network infrastructure plan) 3 Test Plan (DL-05 Test plan)

This file contains only the three S0 plans. The full document set, with the requirements and the 98 test cases, is the Google Doc "OJT-DTR - System Documents - Rev 1".

Revision history

Rev	Date	Change
1	2026-10-21	First issue: module plan, network infrastructure plan, test plan.

1. Modules and Features

1.1 Module map

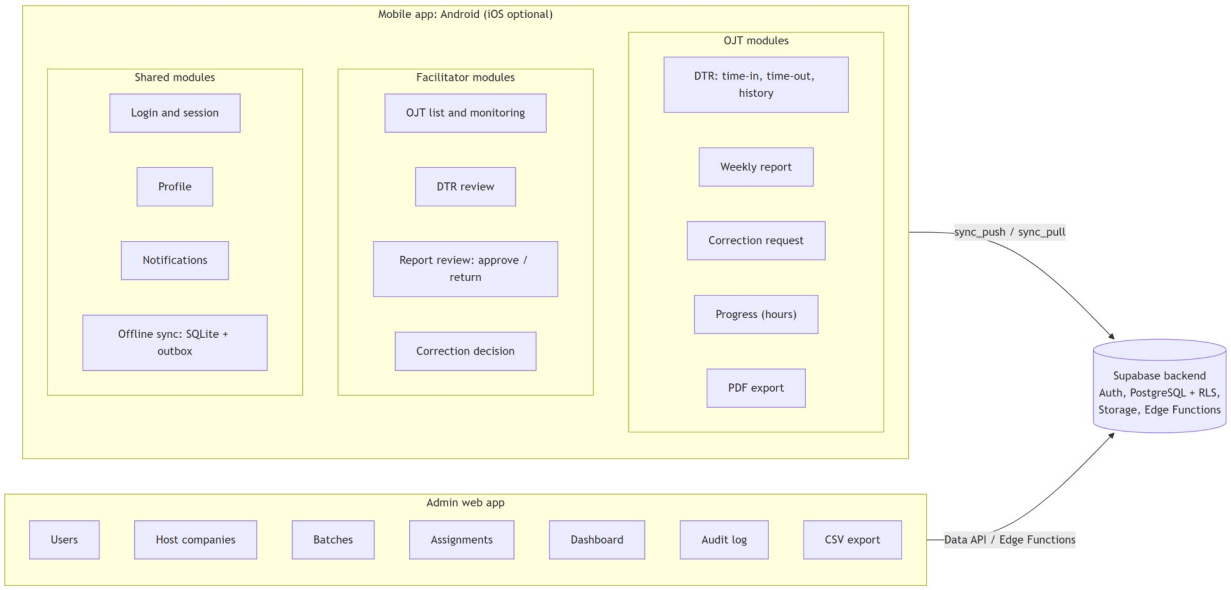


Figure 1. Modules for each role and platform

1.2 Module list

Code	Module	Main features	OJT	Facilitator	Admin	Offline
AUTH	Login and session	Login, routes for each role, token refresh, offline open, logout	Yes	Yes	Yes	Yes, after first login
PROF	Profile	Own profile, host company, facilitator, required hours	Yes	Yes	-	Yes
DTR	Daily time record	Time-in, time-out, location stamp, optional selfie, hours, history	Records	Reviews	Views	Yes
COR	Correction request	Request to add or change a punch. Approve or reject.	Requests	Decides	Audits	Queued
RPT	Weekly report	Draft, attach photos, submit, approve, return with remarks	Writes	Reviews	Views	Draft yes, submit queued
MON	Progress monitoring	Logged, approved, and remaining hours. Late reports.	Own	Assigned OJTs	All	Cached
NOTI	Notifications	Local reminders, push notifications, notification list	Yes	Yes	-	Reminders yes
EXP	Export	DTR and report PDF (mobile), CSV summaries (web)	Own	-	All	PDF yes
ADM	Administration	Users, host companies, batches, assignments, company settings	-	-	Yes	No
AUD	Audit log	Record of approvals, corrections, user and assignment changes	-	-	Yes	N/A

Code	Module	Main features	OJT	Facilitator	Admin	Offline
SYNC	Offline sync	Local database, outbox, sync push and pull, sync issues screen, network status	Yes	Yes	-	Core offline function

1.3 Main flows

Time-in without network

1. The OJT taps "Time in". The app makes sure that no work session is open.
2. The app saves the punch in SQLite with status "pending" and adds an operation to the outbox.
3. The app shows the confirmation at once.
4. When the network is available, the sync engine sends the outbox to the server (sync_push).
5. The server applies each operation one time only. The app sets the punch to "synced".

Weekly report cycle

1. The OJT writes the report (offline is permitted). The app shows the hours of the week from the DTR.
2. The OJT submits the report. The facilitator gets it at the next sync and a push notification.
3. The facilitator approves the report, or returns it with remarks.
4. An approval also approves the DTR entries of that week. The server writes an audit entry.
5. The OJT gets the decision. A returned report can be changed and submitted again.

Correction request

1. The OJT sends a request with a reason to add or change a punch.
2. The facilitator approves or rejects it with a remark.
3. An approval makes a new punch that refers to the original punch. The hours change.

1.4 Capacitor plugins

Plugin	Use
@capacitor-community/sqlite	Local database and outbox (offline functions)
@capacitor/network	Network status, start of sync when the network returns
@capacitor/app	Sync when the app opens again
@capacitor/geolocation	Location stamp on each punch
@capacitor/camera	Selfie on punch (optional), report photos
@capacitor/local-notifications	Time-out and report reminders (offline)
@capacitor/push-notifications	Approval and return messages
@capacitor/filesystem + @capacitor/share	Save and share PDF exports
Secure storage plugin	Safe storage of session tokens

2. Network Infrastructure

The system uses cloud services on free tiers only. The backend is one Supabase project for development and one for production.

2.1 Production topology

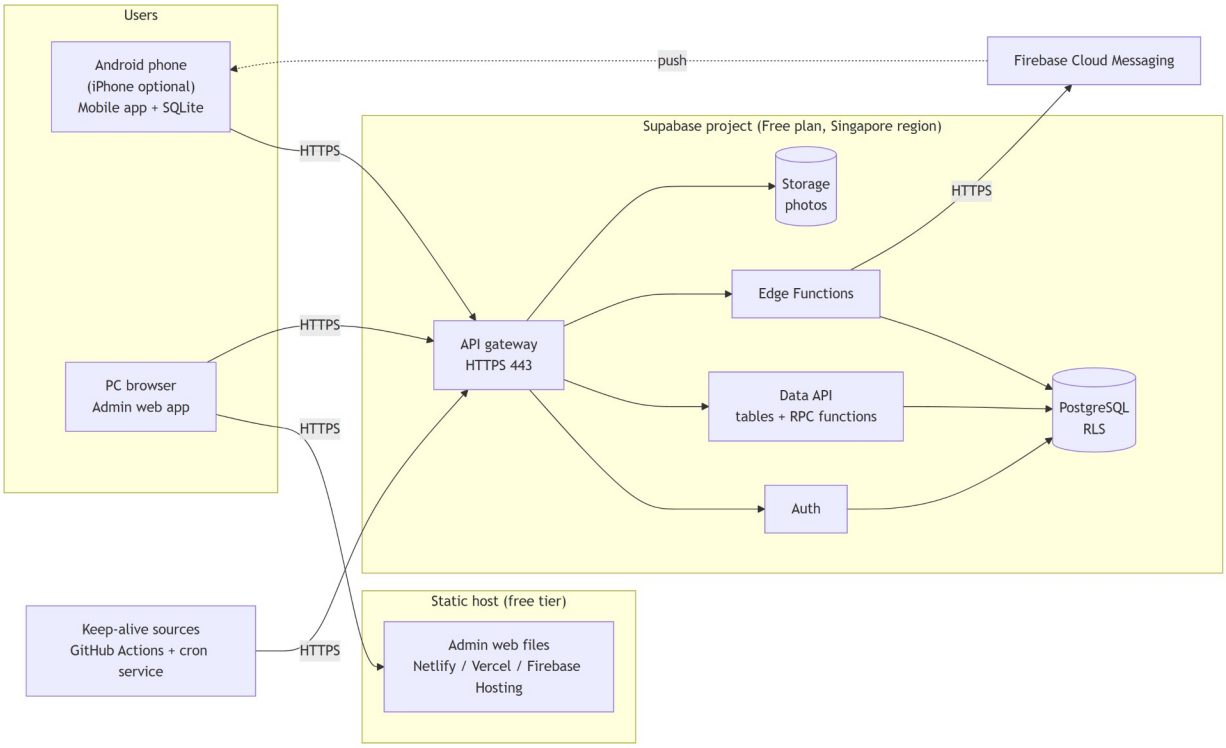


Figure 2. Production topology

2.2 Environments

Environment	Purpose	Backend	Clients
Dev	Development and tests	Supabase project ojt-dtr-dev	Browser, debug APK, emulator
Prod	Demonstration and user acceptance test	Supabase project ojt-dtr-prod	Release APK, admin web URL
Local (optional)	Clean tests and rebuild drill	Supabase CLI + Docker on a laptop	Browser, emulator

2.3 Components

Component	Function	Service
Admin web app	Static files of the Angular web build	Netlify, Vercel, or Firebase Hosting
Auth	Login, sessions, token refresh	Supabase Auth
Data API	Table access and RPC calls (sync_push, sync_pull, heartbeat)	Supabase Data API

Component	Function	Service
Database	PostgreSQL with RLS, functions, triggers	Supabase PostgreSQL
Storage	Photos and attachments (private buckets)	Supabase Storage
Edge Functions	Push messages, admin account actions, CSV export	Supabase Edge Functions
Push service	Push messages to phones	Firebase Cloud Messaging
Keep-alive and backups	Daily heartbeat calls, weekly backup	GitHub Actions, external cron service

2.4 Network flows

#	From	To	Protocol and port	Purpose
F1	Mobile app	Supabase gateway	HTTPS 443	Login, sync, photo upload
F2	Admin browser	Static host	HTTPS 443	Load the admin web app
F3	Admin browser	Supabase gateway	HTTPS 443	Admin data, Edge Functions
F4	Edge Functions	FCM	HTTPS 443	Send push messages
F5	FCM	Mobile device	OS push channel	Deliver push messages
F6	GitHub Actions, cron service	Supabase gateway	HTTPS 443	Heartbeat (keep-alive)
F7	GitHub Actions	Supabase session pooler	PostgreSQL/TLS 5432	Weekly backup
F8	Developer laptop	Supabase	HTTPS 443, 5432	Migrations and deploys

2.5 Security controls

Control	Implementation
Encryption in transit	HTTPS (TLS 1.2 or later) for all traffic
Authentication	Supabase Auth. Token life 15 minutes. Refresh token rotation. Public sign-up off.
Authorization	RLS rules on each table. OJT sees own data. Facilitator sees assigned OJTs only.
Login attempts	Supabase Auth sign-in rate limit
Secrets	Service role key and other secrets only in Edge Function secrets and GitHub secrets. Never in the apps.
Backups	Weekly encrypted own backup

2.6 Free plan limits and protection

Limit	Value	Control
Pause for low activity	After 7 days	Three protection layers (below)
Automatic backups	Not included	Weekly encrypted own backup
Database	500 MB	Enough for one term. Weekly size check.
File storage / egress	1 GB / 5 GB	Photos 500 KB or less, thumbnails, selfie off by default
Active projects	2	dev and prod only

Layer	Measures
1. Prevention	Daily heartbeat from GitHub Actions and from a cron service. Developer opens the apps two times each week. Supabase warning email goes to a read inbox.
2. Tolerance	Apps keep working offline. "Server not reachable" message. No forced sign-out. No data loss.
3. Recovery	Resume a paused project in 30 minutes or less. Rebuild from git and the newest backup in 2 hours or less. Phones send their last 30 days of records again.

2.7 Development network

Phones, the emulator, and the laptop connect to the dev project over HTTPS. A LAN connection between the phone and the laptop is not necessary.

3. Test Plan

3.1 Approach: bottom-up

The team tests the lowest layers first, then adds one layer at a time up to the full system.

- The highest risk is in the low layers: the local database, the outbox, and the sync engine.
- All mobile modules depend on the sync engine and the DTR module. Faults there must be found early.
- The test order agrees with the build order. A UI prototype walkthrough in stage S0 covers the early UI check.

Layer	Contents	Test level
0	Utilities: hours calculation, validation, IDs	Unit
1	Data access: PostgreSQL tables and RLS; SQLite adapter	Unit, component integration
2	Services: database functions, outbox, sync engine, repositories	Unit, component integration
3	Interfaces: Data API, RPC, Edge Functions, route guards	Component integration
4	UI pages for each feature	Component, subsystem integration
5	Full system on devices, web, and cloud	System, acceptance

3.2 Test levels

Level	Purpose	Tools
L1 Unit (U)	Each function is correct alone	Vitest, Angular test runner
L2 Component integration (CI)	Units work together. RLS and functions obey the access rules.	Vitest + supabase-js, SQLite test adapter
L3 Subsystem integration (SI)	App and backend work together on a real device and network	Android phone and emulator, fault injector
L4 System (ST)	Full system meets functional and quality requirements	Playwright, devices, load test, security advisor, accessibility tools
L5 Acceptance (UAT)	Real users can do their work	User scripts, SUS form

3.3 Network conditions for offline tests

Code	Condition
N1	Online, stable
N2	Offline (airplane mode)
N3	Offline to online during work
N4	Online to offline during a sync
N5	Slow network
N6	Network up, backend down or paused
N7	Session expired while offline

3.4 Stage gates

Stage	Dates	Main tests	Exit criteria
S0 Plan and design	Oct 21 to 27	Requirement review, design review, UI walkthrough	Reviews closed. Test plan accepted.
S1 Foundation	Oct 28 to Nov 3	Auth, RLS access, local database, plugin smoke test	Login works for each role. Keep-alive and backups active.
S2 Offline core	Nov 4 to 10	DTR, sync, network fault, app stop, backend down	Zero data loss. No forced sign-out.
S3 Reports and admin	Nov 11 to 17	Report and correction cycles on two devices, admin functions	Access rules pass 100%.
S4 Completion and quality	Nov 18 to 24	All system tests, ISO/IEC 25010 tests, pause and rebuild drills	Each quality target measured.
S5 Release	Nov 25 to 28	Full regression, user acceptance, production smoke test	All "Must" functions pass. Zero open S1 and S2 defects.

Each gate also needs: all planned tests run, zero lint errors, regression suites pass, and a stage test report.

3.5 Defect severity

Severity	Meaning
S1 Critical	Data loss, security breach, or crash in a core flow. No workaround.
S2 Major	A "Must" requirement fails. Workaround is difficult.
S3 Minor	A function fails. An easy workaround exists.
S4 Trivial	Text, layout, or color problem